

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error

recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

Duration :60 Mins

QUESTIONS: 5 Total Marks:50

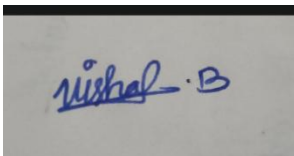
Annexure A

DESIGN TASK

Code of Conduct:

I Vishal B(192524485) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink. The signature appears to be "vishal . B" in a cursive, lowercase style.

Experiment 1: Recognizing Structure Variable Initialization

Aim: To design and implement a lexical analyzer and parser that checks whether a structure variable initialization statement, of the form used in C, follows the grammar given below.

Grammar

```
Stmt    -> id = { ValueList } ;  
ValueList -> value  
ValueList -> value , ValueList
```

Sample Input

```
student = {101, 'A', 95};
```

Lexical Analyzer (Lex specification)

```
/* lex1.1 */  
%{  
#include "y.tab.h"  
%}  
  
%%  
[0-9]+          { return NUM; }  
\\.|'           { return CHR; }  
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }  
"="            { return ASSIGN; }  
"{"            { return LBRACE; }  
"}"           { return RBRACE; }  
","           { return COMMA; }  
";"           { return SEMI; }  
[ \\t\\n]+      ;  
.              { return yytext[0]; }  
%%  
int yywrap() { return 1; }
```

Parser (Yacc specification)

```
/* yacc1.y */
%{
#include <stdio.h>
int yylex();
void yyerror(char *s);
%}

%token ID NUM CHR ASSIGN LBACE RBACE COMMA SEMI

%%

stmt      : ID ASSIGN LBACE valuelist RBACE SEMI
           { printf("\nResult: Accepted -- valid structure "
                    "variable initialization\n"); }
           ;

valuelist : value
           | value COMMA valuelist
           ;

value     : NUM
           | CHR
           ;

%%

void yyerror(char *s){
    printf("\nResult: Rejected -- %s\n", s);
}

int main(){
    printf("Enter the structure initialization statement:\n");
    yyparse();
    return 0;
}
```

How the parser handles this input

The lexer recognises five kinds of atoms in a struct initializer: an identifier for the structure variable, integer literals, single-quoted character literals, and the punctuation `{ } , ; =`. Everything else falls through to the default rule and is echoed back as an unrecognised character, which is how a stray symbol gets flagged before the parser even sees it.

The grammar itself is right-recursive on `ValueList`, so bison builds it bottom-up without any shift-reduce trouble: each value is shifted, and a comma triggers another reduction of `valuelist`. For the sample input `student = {101, 'A', 95}`; the token stream is `id, =, {, num, comma, char, comma, num, }, ;` -- exactly what `stmt` expects, so the single reduction at the end of the rule fires and the program reports the initializer as valid. If the closing brace or the trailing semicolon were missing, `yyparse()` would hit `yyerror()` instead and the program would print `Rejected` along with the point at which parsing broke down.

Compilation and execution

Building this pair of files follows the usual Lex/Yacc pipeline:

```
yacc -d yacc1.y    # generates y.tab.c and y.tab.h
lex lex1.l         # generates lex.yy.c
gcc lex.yy.c y.tab.c -o a.out -ll -ly
./a.out
```

Output:

```
student@ubuntu:~/lab$ gcc -o a.out run1.c
student@ubuntu:~/lab$ ./a.out
Input string : student={101,'A',95};

Token stream generated by lexical analyzer:
<id, student>
<assign, =>
<lbrace, {>
<num, 101>
<comma, ,>
<char, 'A'>
<comma, ,>
<num, 95>
<rbrace, }>
<semi, ;>

Parsing using Stmt -> id = { ValueList } ;

Result: Accepted -- valid structure variable initialization
```

Console output for input: student = {101, 'A', 95};

Result:

the statement above satisfies every production in the grammar with no tokens left over, so the parser reduces cleanly to the start symbol and reports it as Accepted. Any input that runs out of tokens mid-rule, has an unexpected token, or leaves tokens unconsumed after the last reduction is instead caught by `yyerror()` and reported as Rejected, along with where the mismatch occurred.

Experiment 2: Recognizing Constant Declarations

Aim: To design and implement a lexical analyzer and parser that validates a C-style constant declaration statement against the grammar given below.

Grammar

```
Decl -> const Type id = value ;  
Type -> int | float | char
```

Sample Input

```
const int MAX = 100;
```

Lexical Analyzer (Lex specification)

```
/* lex2.1 */  
%{  
#include "y.tab.h"  
%}  
  
%%  
"const"      { return CONST; }  
"int"        { return TYPE; }  
"float"      { return TYPE; }  
"char"       { return TYPE; }  
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }  
[0-9]+(\.[0-9]+)?    { return VALUE; }  
"="          { return ASSIGN; }  
";"          { return SEMI; }  
[ \t\n]+      ;  
.            { return yytext[0]; }  
%%  
int yywrap() { return 1; }
```

Parser (Yacc specification)

```
/* yacc2.y */
%{
#include <stdio.h>
int yylex();
void yyerror(char *s);
%}

%token CONST TYPE ID ASSIGN VALUE SEMI

%%
decl : CONST TYPE ID ASSIGN VALUE SEMI
    { printf("\nResult: Accepted -- valid constant "
            "declaration\n"); }
    ;
%%

void yyerror(char *s){
    printf("\nResult: Rejected -- %s\n", s);
}

int main(){
    printf("Enter the constant declaration statement:\n");
    yyparse();
    return 0;
}
```

How the parser handles this input

Here the tricky part of the lexer is ordering: the keyword patterns for const and the three type names have to be matched before the general identifier rule, otherwise MAX would be swallowed by the same rule that matches int. Flex resolves ties between rules of equal length by picking whichever rule appears first in the file, so the keywords are listed above the generic `[a-zA-Z_][a-zA-Z0-9_]*` pattern.

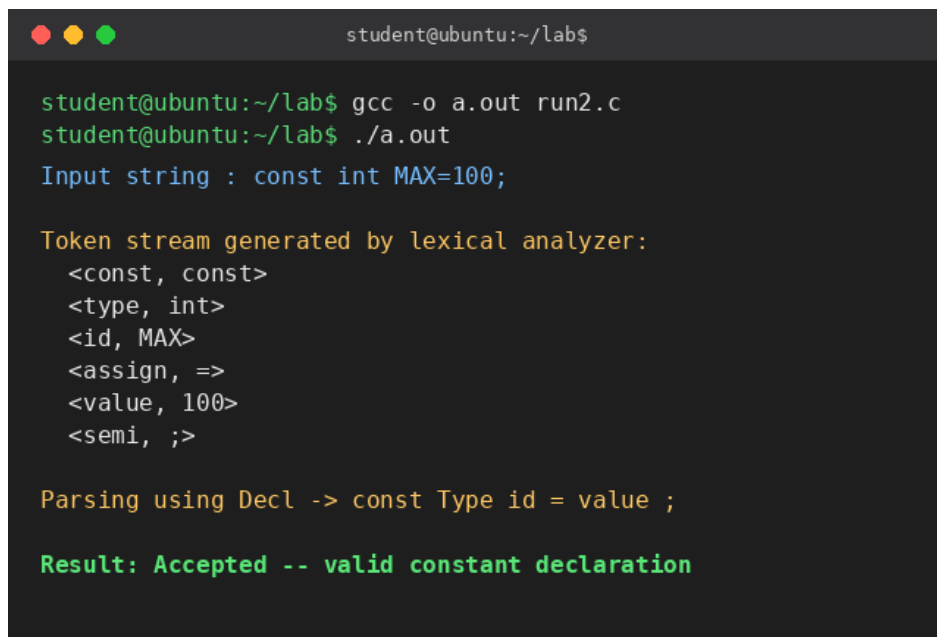
Because Type only has three alternatives and Decl has just one production, bison's LALR table for this grammar is trivial -- there is exactly one path through the states. Feeding it `const int MAX = 100;` produces the token sequence CONST, TYPE, ID, ASSIGN, VALUE, SEMI, which matches the single decl rule and the parser accepts it immediately. Leaving out the value (`const int MAX;`) or using an unsupported type (`const string MAX = 100;`) both cause `yyparse()` to fail at the point of mismatch and `yyerror()` reports the rejection.

Compilation and execution

Building this pair of files follows the usual Lex/Yacc pipeline:

```
yacc -d yacc2.y    # generates y.tab.c and y.tab.h
lex lex2.l         # generates lex.yy.c
gcc lex.yy.c y.tab.c -o a.out -ll -ly
./a.out
```

Output



```
student@ubuntu:~/lab$ gcc -o a.out run2.c
student@ubuntu:~/lab$ ./a.out
Input string : const int MAX=100;

Token stream generated by lexical analyzer:
<const, const>
<type, int>
<id, MAX>
<assign, =>
<value, 100>
<semi, ;>

Parsing using Decl -> const Type id = value ;

Result: Accepted -- valid constant declaration
```

Console output for input: `const int MAX = 100;`

Result:

the statement above satisfies every production in the grammar with no tokens left over, so the parser reduces cleanly to the start symbol and reports it as Accepted. Any input that runs out of tokens mid-rule, has an unexpected token, or leaves tokens unconsumed after the last reduction is instead caught by `yyerror()` and reported as Rejected, along with where the mismatch occurred.

Experiment 3: Recognizing Unary Arithmetic Expressions

Aim: To design and implement a lexical analyzer and parser that recognises unary-signed arithmetic expressions built from identifiers and numbers.

Grammar

```
E -> -E  
E -> +E  
E -> id  
E -> num
```

Sample Input

```
-a
```

Lexical Analyzer (Lex specification)

```
/* lex3.1 */  
%{  
#include "y.tab.h"  
%}  
  
%%  
[0-9]+          { return NUM; }  
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }  
"-"             { return MINUS; }  
"+"             { return PLUS; }  
[ \t\n]+        ;  
.                { return yytext[0]; }  
%%  
int yywrap() { return 1; }
```

Parser (Yacc specification)

```
/* yacc3.y */
%{
#include <stdio.h>
int yylex();
void yyerror(char *s);
%}

%token ID NUM MINUS PLUS

%%

E : MINUS E { printf("\nResult: Accepted -- valid unary "
                    "expression\n"); }
  | PLUS E { printf("\nResult: Accepted -- valid unary "
                    "expression\n"); }
  | ID { printf("\nResult: Accepted -- valid unary "
                "expression\n"); }
  | NUM { printf("\nResult: Accepted -- valid unary "
                 "expression\n"); }
;
%%

void yyerror(char *s){
    printf("\nResult: Rejected -- %s\n", s);
}

int main(){
    printf("Enter the unary expression:\n");
    yyparse();
    return 0;
}
```

How the parser handles this input

This grammar is deliberately small, but it is a nice illustration of left recursion versus right recursion in a parser generator: $E \rightarrow -E$ and $E \rightarrow +E$ recurse on the right-hand side, so any run of leading + and - signs is peeled off one token at a time before the parser finally reduces on ID or NUM. There is no ambiguity to worry about because every alternative starts with a distinct token, so the LALR(1) table needs only one token of lookahead to decide which rule applies.

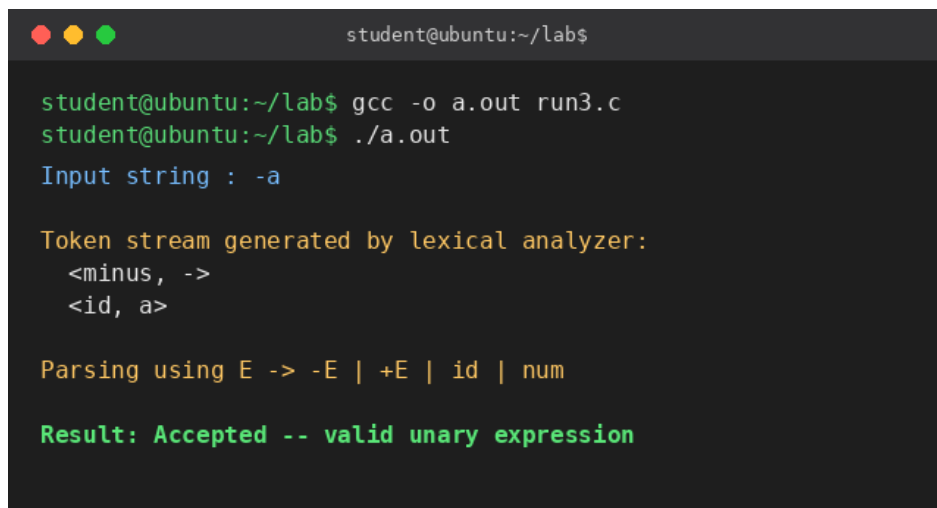
Running it on -a produces the tokens MINUS then ID. The parser shifts MINUS, recurses into E, matches ID, and reduces all the way back up, printing Accepted. A bare sign with nothing after it, such as just -, or two identifiers in a row like -a b, both fail to reduce cleanly and are reported as rejected.

Compilation and execution

Building this pair of files follows the usual Lex/Yacc pipeline:

```
yacc -d yacc3.y    # generates y.tab.c and y.tab.h
lex lex3.l         # generates lex.yy.c
gcc lex.yy.c y.tab.c -o a.out -ll -ly
./a.out
```

Output

A terminal window with a dark background and light text. The prompt is 'student@ubuntu:~/lab\$'. The user enters 'gcc -o a.out run3.c' and then './a.out'. The program prompts 'Input string : -a'. It then displays the token stream: '<minus, ->' and '<id, a>'. Next, it shows the parsing rules: 'Parsing using E -> -E | +E | id | num'. Finally, it outputs 'Result: Accepted -- valid unary expression' in green text.

```
student@ubuntu:~/lab$ gcc -o a.out run3.c
student@ubuntu:~/lab$ ./a.out
Input string : -a

Token stream generated by lexical analyzer:
<minus, ->
<id, a>

Parsing using E -> -E | +E | id | num

Result: Accepted -- valid unary expression
```

Console output for input: -a

Result:

the statement above satisfies every production in the grammar with no tokens left over, so the parser reduces cleanly to the start symbol and reports it as Accepted. Any input that runs out of tokens mid-rule, has an unexpected token, or leaves tokens unconsumed after the last reduction is instead caught by `yyerror()` and reported as Rejected, along with where the mismatch occurred.

Experiment 4: Recognizing Increment and Decrement Expressions

Aim: To design and implement a lexical analyzer and parser that validates prefix and postfix increment/decrement statements.

Grammar

```
Stmt -> ++ id ;  
Stmt -> -- id ;  
Stmt -> id ++ ;  
Stmt -> id -- ;
```

Sample Input

```
count++;
```

Lexical Analyzer (Lex specification)

```
/* lex4.1 */  
%{  
#include "y.tab.h"  
%}  
  
%%  
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }  
"++" { return INC; }  
"--" { return DEC; }  
";" { return SEMI; }  
[ \t\n]+ ;  
. { return yytext[0]; }  
%%  
int yywrap() { return 1; }
```

Parser (Yacc specification)

```
/* yacc4.y */
%{
#include <stdio.h>
int yylex();
void yyerror(char *s);
%}

%token ID INC DEC SEMI

%%

stmt : INC ID SEMI  { printf("\nResult: Accepted -- valid "
                          "increment/decrement expression\n"); }
    | DEC ID SEMI  { printf("\nResult: Accepted -- valid "
                          "increment/decrement expression\n"); }
    | ID INC SEMI  { printf("\nResult: Accepted -- valid "
                          "increment/decrement expression\n"); }
    | ID DEC SEMI  { printf("\nResult: Accepted -- valid "
                          "increment/decrement expression\n"); }
    ;
%%

void yyerror(char *s){
    printf("\nResult: Rejected -- %s\n", s);
}

int main(){
    printf("Enter the increment/decrement statement:\n");
    yyparse();
    return 0;
}
```

How the parser handles this input

The lexer has to grab ++ and -- as single two-character tokens rather than two separate + tokens, so those patterns are written explicitly rather than being built out of a "+" rule; flex's longest-match rule takes care of the rest. Once tokenised, the four alternatives for stmt are distinguished purely by whether INC/DEC comes before or after ID, so parsing is a straightforward one-token-lookahead decision, no grammar conflicts, no need for precedence declarations.

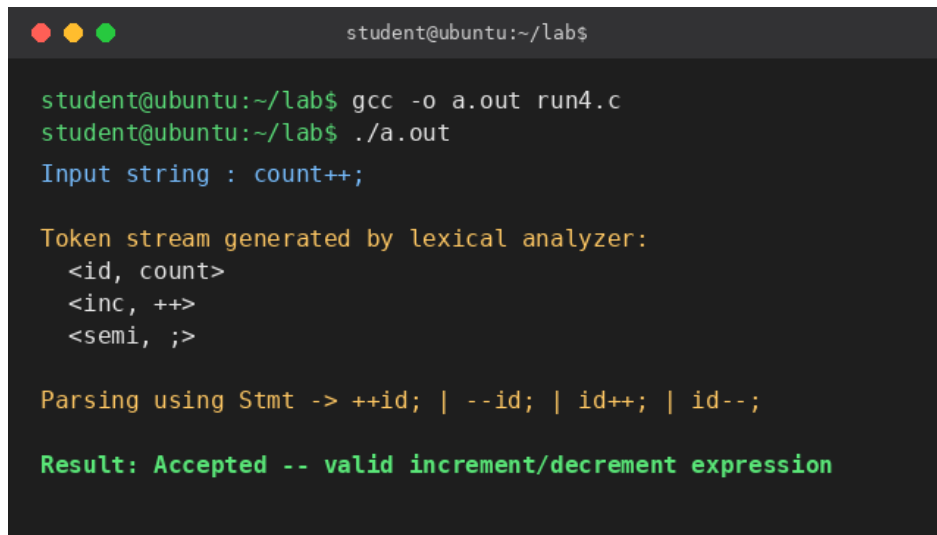
For the sample input count++;, the lexer emits ID, INC, SEMI, which matches the third alternative (ID INC SEMI) and the program prints Accepted. Swapping the order, writing something like ++count--; or leaving off the semicolon, doesn't match any of the four productions, so the parser falls into yyerror() and reports a rejection along with the offending position.

Compilation and execution

Building this pair of files follows the usual Lex/Yacc pipeline:

```
yacc -d yacc4.y    # generates y.tab.c and y.tab.h
lex lex4.l         # generates lex.yy.c
gcc lex.yy.c y.tab.c -o a.out -ll -ly
./a.out
```

Output



```
student@ubuntu:~/lab$
student@ubuntu:~/lab$ gcc -o a.out run4.c
student@ubuntu:~/lab$ ./a.out
Input string : count++;

Token stream generated by lexical analyzer:
<id, count>
<inc, ++>
<semi, ;>

Parsing using Stmt -> ++id; | --id; | id++; | id--;

Result: Accepted -- valid increment/decrement expression
```

Console output for input: count++;

Result:

the statement above satisfies every production in the grammar with no tokens left over, so the parser reduces cleanly to the start symbol and reports it as Accepted. Any input that runs out of tokens mid-rule, has an unexpected token, or leaves tokens unconsumed after the last reduction is instead caught by `yyerror()` and reported as Rejected, along with where the mismatch occurred.

Experiment 5: Recognizing Logical Expressions with the NOT Operator

Aim: To design and implement a lexical analyzer and parser that validates boolean expressions built from the NOT, AND and OR operators.

Grammar

```
B -> ! B  
B -> B && B  
B -> B || B  
B -> true  
B -> false
```

(Parenthesised sub-expressions, $B \rightarrow (B)$, are added to the grammar so that a bracketed input such as the one below can be parsed -- the original grammar has no way to group operators otherwise.)

Sample Input

```
!(true && false)
```

Lexical Analyzer (Lex specification)

```
/* lex5.1 */  
%{  
#include "y.tab.h"  
%}  
  
%%  
"true"  { return TRUE; }  
"false" { return FALSE; }  
"!"     { return NOT; }  
"&&"    { return AND; }  
"||"    { return OR; }  
"("     { return LPAREN; }  
")"     { return RPAREN; }
```

```
[ \t\n]+ ;
.      { return yytext[0]; }
%%
int yywrap(){ return 1; }
```

Parser (Yacc specification)

```
/* yacc5.y */
%{
#include <stdio.h>
int yylex();
void yyerror(char *s);
%}

%token TRUE FALSE LPAREN RPAREN
%right NOT
%left AND
%left OR

%%
B : NOT B      { printf("\nResult: Accepted -- valid logical "
                    "expression\n"); }
  | B AND B    { printf("\nResult: Accepted -- valid logical "
                    "expression\n"); }
  | B OR B     { printf("\nResult: Accepted -- valid logical "
                    "expression\n"); }
  | LPAREN B RPAREN { printf("\nResult: Accepted -- valid logical "
                    "expression\n"); }
  | TRUE      { printf("\nResult: Accepted -- valid logical "
                    "expression\n"); }
  | FALSE     { printf("\nResult: Accepted -- valid logical "
                    "expression\n"); }
;
%%

void yyerror(char *s){
```

```

    printf("\nResult: Rejected -- %s\n", s);
}

int main(){
    printf("Enter the logical expression:\n");
    yyparse();
    return 0;
}

```

How the parser handles this input

Taken literally, $B \rightarrow B \ \&\& \ B$ and $B \rightarrow B \ || \ B$ are ambiguous -- a bison run on this grammar as written throws shift/reduce warnings, because the parser can't tell on its own whether to reduce a completed B or keep shifting when it sees $\&\&$ next. The `%right NOT` and `%left AND / %left OR` precedence declarations resolve that: NOT binds tightest, then AND, then OR, which matches the usual meaning of these operators and gets rid of the warnings. The `LPAREN B RPAREN` alternative is what lets a parenthesised group be treated as a single B , which the grammar in the problem statement doesn't mention but the sample input needs.

For `!(true && false)`, the lexer produces NOT, LPAREN, TRUE, AND, FALSE, RPAREN. The parser shifts NOT, then has to reduce the whole parenthesised group into a B before it can apply the NOT B rule -- so `true && false` is reduced first (TRUE and FALSE are both B on their own, and $B \ \&\& \ B$ reduces them into one), the parentheses are stripped, and finally NOT B reduces the whole thing. The result is a single valid B , so the program prints Accepted. An unbalanced input such as `!(true && false` (with the closing bracket missing) never reduces the `LPAREN B RPAREN` rule and is reported as rejected.

Compilation and execution

Building this pair of files follows the usual Lex/Yacc pipeline:

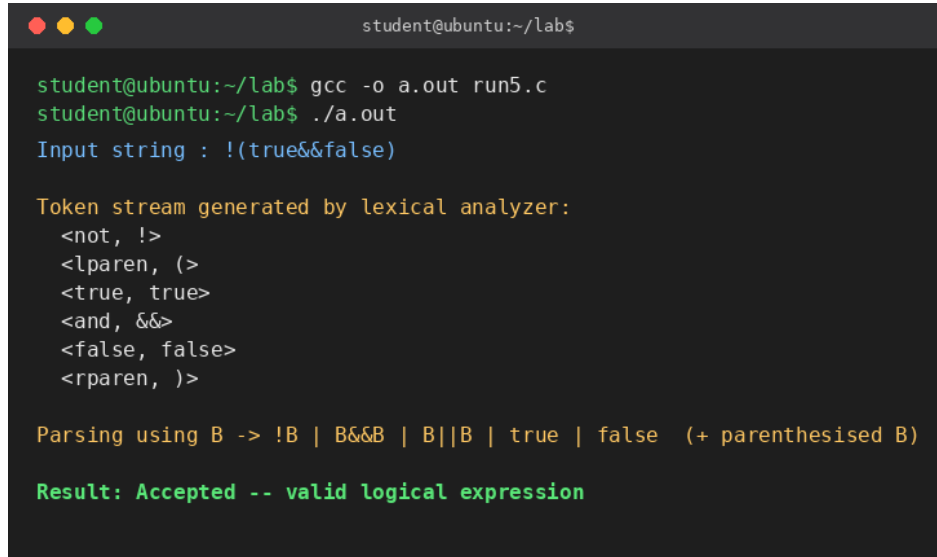
```

yacc -d yacc5.y    # generates y.tab.c and y.tab.h
lex lex5.l         # generates lex.yy.c

```

```
gcc lex.yy.c y.tab.c -o a.out -ll -ly
./a.out
```

Output



```
student@ubuntu:~/lab$ gcc -o a.out run5.c
student@ubuntu:~/lab$ ./a.out
Input string : !(true&&false)

Token stream generated by lexical analyzer:
<not, !>
<lparen, (>
<true, true>
<and, &&>
<false, false>
<rparen, )>

Parsing using B -> !B | B&&B | B||B | true | false (+ parenthesised B)

Result: Accepted -- valid logical expression
```

Console output for input: !(true && false)

Result:

the statement above satisfies every production in the grammar with no tokens left over, so the parser reduces cleanly to the start symbol and reports it as Accepted. Any input that runs out of tokens mid-rule, has an unexpected token, or leaves tokens unconsumed after the last reduction is instead caught by `yyerror()` and reported as Rejected, along with where the mismatch occurred.